Tutorial: The Chaos Game and Vectorisation

Matteo Fasiolo

The Chaos Game

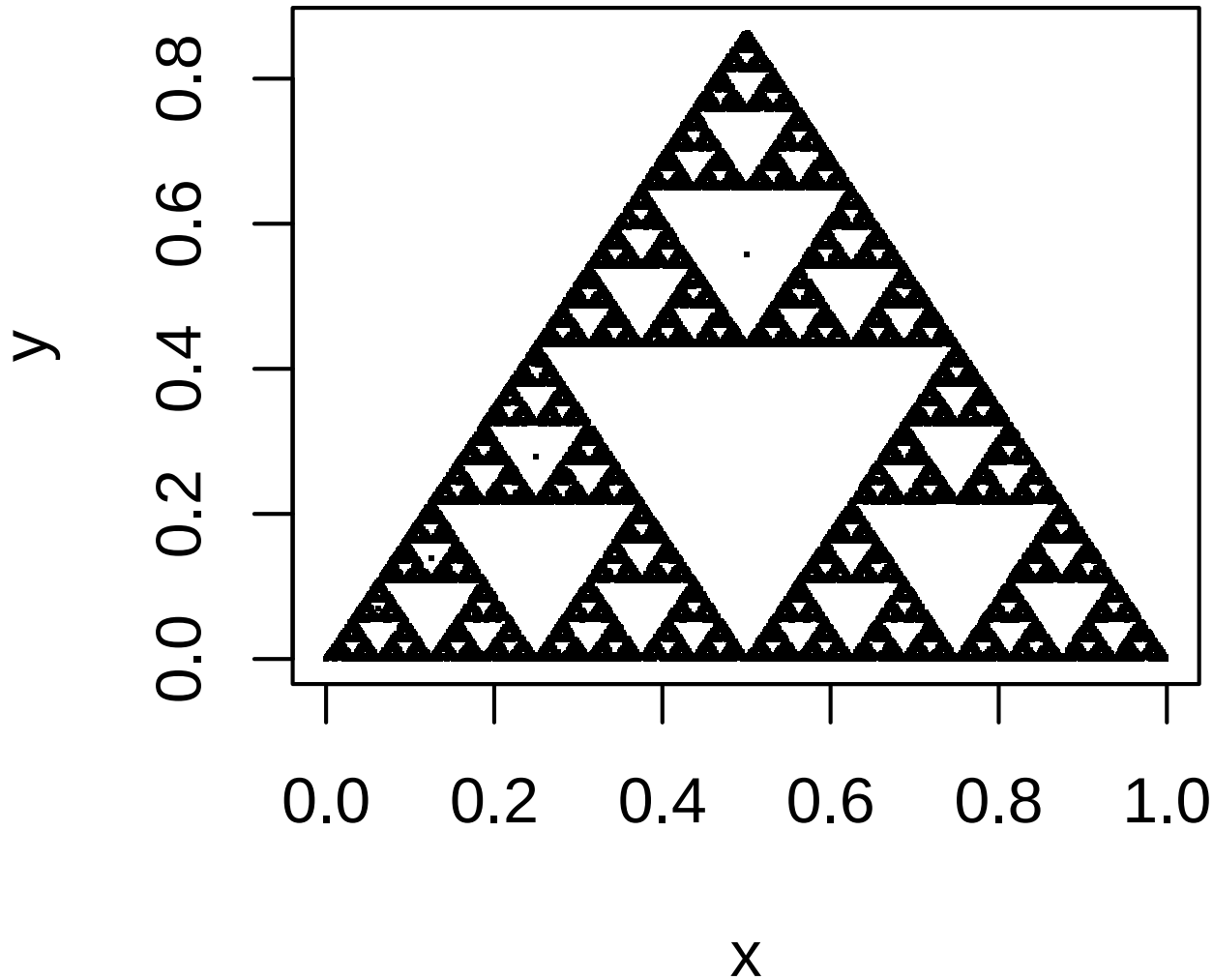
Here is a surprising game that draws a beautiful picture from a very simple rule.

Take the three corners of a triangle. Start somewhere inside it. Then repeat, thousands of times:

1. pick one of the three corners at random;
2. move to the point half-way between where you are now and that corner;
3. put a dot at your new position.

You would expect a random smudge. Instead you get this...

The target picture



This is the **Sierpinski triangle**. Your job today is to draw it, and then to make your code fast.

The plan

The picture is the fun part, but it is the easy part.

The real goal of this tutorial is a slow R program fast by:

- **pre-allocating** storage instead of growing a vector, and
- moving repeated work **out of the loop** and **vectorising** it.

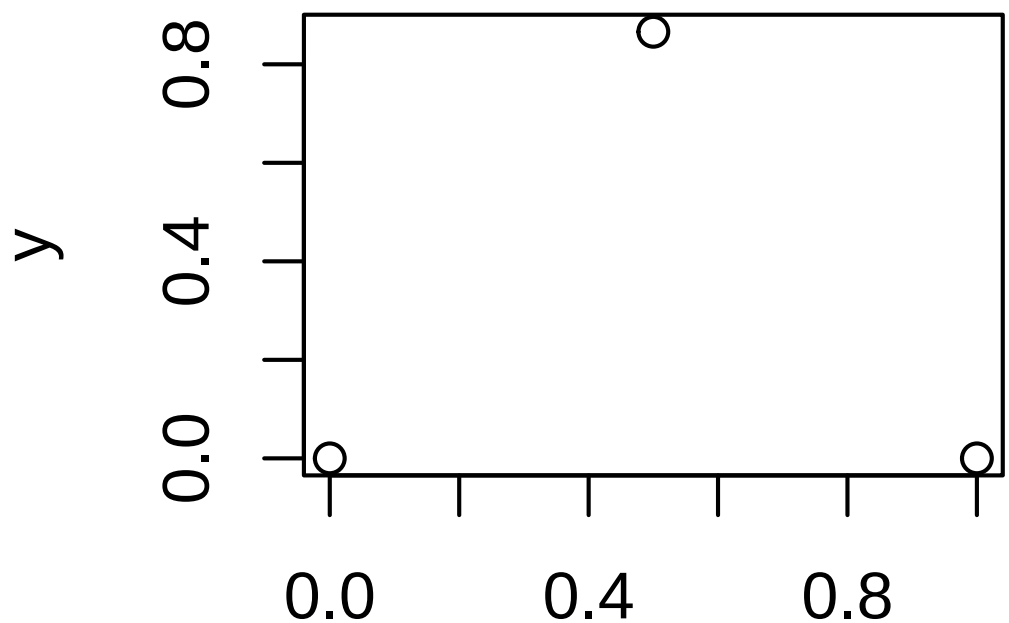
Keep the lecture slides on "Speeding up R" ready for reference.

Step 1: the corners

We store the three corners as the rows of a 3×2 matrix. Row `k` holds the (x, y) coordinates of corner `k`.

```
V <- matrix(c(0,   0,             # corner 1
              1,   0,             # corner 2
              0.5, sqrt(3)/2),    # corner 3
            nrow = 3, ncol = 2, byrow = TRUE)

plot(V[, 1], V[, 2], xlab = "x", ylab = "y")
```



Step 2: a first (slow) version

Complete the skeleton below. Each step picks a random corner `k`, moves half-way to it, and **stores** the new point. To turn a random `u` in $[0, 1]$ into a corner 1, 2 or 3 we use `as.integer(u * 3) + 1`.

```
chaos_slow <- function(n){
  x <- 0.5; y <- 0.25
  xs <- c(); ys <- c()
  for(i in 1:n){
    u <- runif(1)
    k <- as.integer(u * 3) + 1  # corner 1, 2, 3
    # TODO: move half-way to corner k (x, y)
    # and grow x and xs one element at the time
  }
  return(cbind(xs, ys))
}
```

Draw it

Once your function works, draw the result with a single `plot()`:

```
pts <- chaos_slow(3e4)
# TODO: Call plot here!
```

Questions

1. Complete `chaos_slow` and check that you obtain the Sierpinski triangle.
2. In `plot()`, try `pch = "."` versus the default. Which is better for 30 000 points?

Step 3: how fast is it?

Time your function for a few values of `n`, using `Sys.time()`:

```
for(n in c(2000, 20000, 100000)){
 # TODO Your code here
}
```

Questions

4. Roughly, when you multiply `n` by 10, does the time multiply by about 10, or by a lot more?
5. A run with `n = 1e5` may take many seconds. That is shocking for such a tiny calculation. Where is all the time going? Think about this before turning page.

Why is it slow?

Two things are wrong, and both were warned about in lectures.

Problem 1: growing a vector. Code such as:

```
xs <- c(xs, x)
```

does not just add one number. Each time, R makes a brand-new, longer vector and copies everything across. Over `n` steps that is roughly $1 + 2 + \cdots + n \approx n^2/2$ copies: the cost grows like n^2.

Problem 2: calling `runif(1)` n times. Each call has an overhead. Calling `runif` once for a single number, `n` times, wastes most of that effort.

Fix 1: pre-allocate the storage

Instead of growing `xs` and `ys`, construct the result matrix once, at full size, and fill-in row `i` at step `i`.

```
chaos_prealloc <- function(n){
  x <- 0.5; y <- 0.25

  # TODO create a matrix "pts" to store all the
  # n points.
  # Modify the function to store the points in
  # pts, rather than growing xs and ys.
  return(pts)
}
```

Question

6. Time `chaos_prealloc` against `chaos_slow` at `n = 1e5`. How big is the speed-up?

Fix 2: vectorise the randomness

We still call `runif(1)` inside the loop `n` times. But we can make all the choices in one call, before the loop, for example:

```
choices <- as.integer(runif(6) * 3) + 1   # 6 draws
choices
```

```
[1] 2 1 3 2 2 3
```

Also, we can then look up all the chosen corners at once, because a matrix can be indexed by a whole vector of row numbers:

```
V[choices, ]   # one row per chosen corner
```

```
     [,1]      [,2]
[1,]  1.0 0.0000000
[2,]  0.0 0.0000000
[3,]  0.5 0.8660254
[4,]  1.0 0.0000000
[5,]  1.0 0.0000000
```

Fix 2: the fast version

```
chaos_fast <- function(n){

  # TODO modify chaos_prealloc to make all the
  # random choices in one go, and look up
  # all the corners in one go.

  return(pts)
}
```

Questions

7. Complete `chaos_fast` and check it still draws the triangle.
8. Time all three versions at `n = 1e5`.

What we can, and cannot, vectorise

Notice we could not delete the `for` loop entirely.

Each new point depends on the previous one:

$$p_i = \tfrac{1}{2}\left(p_{i-1} + \text{corner}\right).$$

This is a data dependency: point `i` cannot be computed until point `i-1` is known, so the steps must happen in order.

The lesson is: **vectorise everything you can** (here: the storage and the random draws). Here one genuinely sequential loop has to remain.

Step 4: a fully vectorised measurement

Some questions about the finished picture need no loop at all. Suppose `pts` is your matrix of points.

Questions

9. Using a **single line, no loop**, count how many points have `x`-coordinate less than `0.5`.
   - Hint: recall that `sum()` is vectorised.
10. Using a **single line, no loop**, compute the Euclidean distance from every point to corner 1, which is at the origin $(0, 0)$.

Challenges

11. **A different ratio.** Replace the factor $1/2$ by a factor `ratio` (so `x <- x + ratio * (V[k,1] - x)`). Try `ratio` of 0.4 and of 0.6. How does the picture change?

12. **Four corners.** Put the four corners of a square in `V` and pick one of four at random. Does a square Sierpinski pattern appear? (It may not, experiment!)

13. **Colour by corner.** In `chaos_fast` you already have the vector `choices`. Colour each point by the corner it jumped to.

Summary

- Try to avoid growing a vector with `c()` inside a loop: pre-allocate with `matrix()` or `numeric()`.
- Try to pull repeated work out of the loop and do it in one vectorised call (here, `runif(n)` and a vectorised matrix lookup).
- Some loops are genuinely sequential and cannot be removed, but you can still vectorise everything around them.